

[과제] NLP 기초: CountVectorizer를 이용한 뉴스 그룹 대화 분류(코사인 유사성)

1. 과제 개요

본 과제에서는 **sklearn** 라이브러리의 뉴스그룹 데이터셋을 활용하여 텍스트 데이터를 수치화하고, 코사인 유사도를 통해 새로운 문장이 어떤 주제에 속하는지 분류하는 프로그램을 작성합니다.

2. 필수 요구 사항

- 데이터 수집: **fetch_20newsgroups**를 사용하여 3가지 주제(**comp.graphics**, **sci.space**, **talk.religion.misc**)를 선택하고, 각 주제당 **20**개의 샘플 데이터를 추출하세요.
- 텍스트 전처리: 뉴스 데이터의 헤더, 푸터, 인용구(**headers**, **footers**, **quotes**)를 제거하여 순수 본문만 사용하세요.
- 벡터화: **CounterVectorizer**를 사용하여 텍스트를 빈도 기반 벡터로 변환하세요. (이때 영어 불용어 제거 기능을 포함하세요.)
- 유사도 계산: **cosine_similarity**를 사용하여 입력된 문장과 기존 데이터 간의 유사도를 측정하세요.

3. 실습 코드 (템플릿)

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# [Step 1] 데이터 로드 및 샘플링
categories = ['comp.graphics', 'sci.space', 'talk.religion.misc']
newsgroups = fetch_20newsgroups(subset='train', categories=categories,
                                remove=('headers', 'footers', 'quotes'))

data, labels = [], []
for i, category in enumerate(categories):
    indices = np.where(newsgroups.target == i)[0][:100] # 각 주제별 20개 추출
    for idx in indices:
        data.append(newsgroups.data[idx])
        labels.append(newsgroups.target_names[i])
```

```

# [Step 2] CountVectorizer 설정 및 변환
# TODO: CountVectorizer를 선언하고 데이터를 fit_transform 하세요.
vectorizer = CountVectorizer(stop_words='english')
count_matrix = vectorizer.fit_transform(data)

# [Step 3] 분류 함수 구현
def classify_text(input_text):
    # TODO: 입력 텍스트를 벡터화하고 코사인 유사도를 계산하는 로직을 작성하세요.
    input_vec = vectorizer.transform([input_text])
    sim = cosine_similarity(input_vec, count_matrix)

    best_idx = np.argmax(sim)
    return labels[best_idx], sim[0][best_idx]

# [Step 4] 테스트 실행
test_sentences = [
    "The rocket launched into orbit.",
    "A new 3D rendering technique for graphics.",
    "Theological discussions on faith and god."
]

for s in test_sentences:
    cat, score = classify_text(s)
    print(f"문장: {s[:30]}... | 예측: {cat} | 유사도: {score:.4f}")

```

4. 고찰 과제 (보고서 포함 내용)

Q1. 유사도가 **0.0000**이 나오는 이유 분석 특정 문장(예: "Exploring the mars with a robotic rover.")을 입력했을 때 유사도가 0이 나오거나 엉뚱한 카테고리가 매칭되는 이유를 **CountVectorizer**의 동작 원리와 관련지어 설명하세요.

Q2. 성능 개선 실험 유사도 점수를 높이고 분류 정확도를 올리기 위해 다음 중 하나를 시도하고 결과를 비교하세요.

- 주제별 샘플 수를 20개에서 100개로 늘렸을 때의 변화.
 - **CountVectorizer** 대신 **TfidfVectorizer**를 사용했을 때의 차이점.
 - **ngram_range=(1, 2)** 설정을 추가했을 때의 영향.
-



구현 가이드: 뉴스그룹 데이터 분류 시스템

1. 환경 설정 및 데이터 준비

- 라이브러리 импорт: `sklearn` 내 데이터셋, 벡터화 도구, 유사도 계산 함수를 불러옵니다.
- 주제 선정: `categories` 리스트를 만들어 3가지 서로 다른 도메인(예: 그래픽, 우주, 종교)을 지정합니다.
- 데이터 로드: `fetch_20newsgroups` 함수를 사용하되, 노이즈 제거를 위해 `remove=('headers', 'footers', 'quotes')` 옵션을 반드시 적용합니다.
- 데이터 샘플링:
 - 전체 데이터를 다 쓰지 않고 각 주제별로 정확히 20개씩만 추출합니다.
 - 이는 데이터 부족으로 인한 '유사도 0' 현상을 학생들이 직접 경험하게 하기 위함입니다.

2. 텍스트 데이터의 수치화 (Vectorization)

- `CountVectorizer` 객체 생성:
 - `stop_words='english'`를 설정하여 의미 없는 관용구(a, an, the 등)를 분석에서 제외합니다.
- 어휘 사전 학습(Fitting):
 - `fit_transform()` 메서드를 사용하여 추출된 60개(20개 × 3주제)의 문서에 등장하는 단어 사전을 만들고, 문서를 **빈도 기반 행렬(Sparse Matrix)**로 변환합니다.
- 특징량 확인: `vectorizer.get_feature_names_out()`을 통해 생성된 사전에 어떤 단어들이 포함되었는지 확인해 봅니다.

3. 사용자 입력 및 유사도 분석

- 입력 데이터 변환:
 - 사용자가 입력한 문장은 학습된 모델의 `transform()` 메서드만을 사용하여 벡터화합니다.
 - 주의: 새롭게 `fit`을 하면 기존 데이터와 차원(Dimension)이 달라져 비교가 불가능합니다.
- 코사인 유사도(Cosine Similarity) 계산:
 - 입력 문장 벡터와 기존 60개 문서 벡터 간의 각도를 계산합니다.
 - 결과는 `[1, 60]` 형태의 배열로 반환되며, 각 값은 해당 문서와의 유사도(0~1)를 나타냅니다.

4. 결과 판정 및 출력

- 최적 일치 검색: `np.argmax()`를 사용하여 유사도 점수가 가장 높은 인덱스를 찾습니다.
- 임계값 확인:
 - 만약 모든 유사도 점수가 0이라면, 학습 데이터에 입력 단어가 하나도 없음을 의미합니다.
- 최종 매칭: 해당 인덱스의 라벨명(주제명)과 유사도 점수를 화면에 출력합니다.

🚀 코사인 유사성(cosine similarity)



영화대사에서 배크맨과 조커사용하는 대사를 kill, save 단어만으로 countervetorize한다면, 아래 그림을 X축은 save 단어 출현횟수 Y축은 kill 단어의 출현회수로 모든 배트맨과 조커의 모든 대사를 데이터포인트한다면?

Cosine Similarity

